

one

Bash

Terminal & Finder

Terminal works as a **REPL**

- R : Read
- E : Eval
- P : Print
- L : Loop

Some Commands

```
echo
pwd
ls
touch file.txt -> Make a file
rm file.txt -> remove a file
clear
cd
```

File Manipulation

```
> touch hello.txt
> touch hello12.txt
> mv hello.txt hello1.txt
> mv hello1.txt hello12.txt
#mv command will rename or move without giving any errors
#4th command works without giving any errors
> rm *.txt
rm -i #for interactive mode -> asks you what to remove -> `makes it less dangerous`
alias rm='rm -i'
```

history -> get history of the bash

hidden files

- file that start with . is a hidden file
- `ls -a` -> gives you all the files
- `'.'` -> current directory
- `'..'` -> directory one level above
- `'-'` -> previous directory

```
cd downloads
cd ~/documents
cd - # gets you back into in the downloads
```

searching in files

cat -> open your files

concat

```
cat hello.txt
#everything inside the hello.txt is printed to the terminal
```

grep -> another tool which is used for the finding patterns in a input stream or file

```
grep 'hello' /usr/share/dict/words
#everything having 'hello' in it

grep '^hello' /usr/share/dict/words
#everything starting with 'hello' in it

grep 'hello$' /usr/share/dict/words
#everything ending with 'hello' in it
```

```
echo hello > file.txt
#overwrites the whole file and write

echo hello >> file.txt
#appends to the file.txt

grep -A1 hello file.txt
#find hello and get me the one line after it

grep -B1 hello file.txt
#find hello and get me the one line before it

grep -C1 hello file.txt #C:→ context
#find hello and get me the one line before and after it

grep -i hello file.txt #i:→ insensitive
#find hello irrespective of the case

grep -o hello file.txt #o:→ only the part that has matched the pattern

#we can also combine these
grep -io hello file.txt
grep -oi hello file.txt
grep -i -o hello file.txt
grep -o -i hello file.txt
## all of these are the same things

#use grep to get the count of the words
grep -c d /usr/share/dict
```

pipelines

```
cat file.txt | grep hello
#if we don't provide a input grep reads from its standard input
#all the data of the file.txt is parsed to grep and grep searches for hello it is the same as
`grep hello file.txt`
```

paging files

making a large output of a command readable

- 'less'
- 'more'

less is more powerful than **more**

```
less cat /usr/share/dict/word
more cat /usr/share/dict/word
#these do the same thing

cat /usr/share/dict/word | less
#piping to the less
```

man pages

manual pages

```
man mv
# manual for the mv
# three important sections
# 1 :- General commands `mv` `cp` `ls`
# 2 :- System Calls Manual
# 3 :- Library Functions manual `std::vector`

man 1 cp
man printf → printf for the terminal
man 3 printf → printf from the C
```

- man equivalent **history**

```
help history #→ for the things built into the shell
```

- type command

```
type history
> history is a shell builtin

type -a ls
# everything from the top to bottom
# alias ls='ls -p --color=auto'
# ls
# so the type command would run all the options or the alias before going to bed
```

We need to use help command for the built in types

Programs and commands

file -> tells us the type of the program

Q) when you typed 'ls' how did the bash knew where to look?

ans: it was already mentioned in the path

```
echo $PATH
```

 we used \$ in-order to instruct the bash to expand the variable

```
echo $PATH
```

```
echo $PATH | tr : '\n'
```

```
#tr → translate command
```

```
#changes the ':' → '\n'
```

tr command

```
echo "$PATH" | tr : '\n' | tr 'a' 'b'
```

```
# replace ':' → tr and 'a' → 'b'
```

```
echo "$PATH" | tr :a '\nb'
```

```
# same as the first one
```

```
echo "$PATH" | tr abc 'xyz'
```

```
# a → x
```

```
# b → y
```

```
# c → z
```

some more variables

Use \$ to expand these

- PWD
- USER
- SHELL
- HOSTNAME
- MACHTYPE -> machine type
- uname

defining a variable

```
name='Medhansh'
```

```
#if i want to have a dyanaminc name
```

```
name=`uname -a`
```

```
#C equivalent printf("%d");
```

```
#this is depricated so we use this
```

```
name=$(uname -a)
```

Vim the editor

commands to activate/open

1. vi
2. vim
3. nvim

```
sudo dnf install nvim  
sudo apt install nvim
```

to open `vim file.txt`

three modes of vim

1. normal
2. visual
3. editing

1. normal

to enter this mode

comes by default (When you open the file)

dd -> removes the line (no matter where the cursor is)

. -> replays the last command

p -> paste

yy -> yanking (copying)

u -> undo

G -> end of the file (shift + g)

gg -> top of the file

2. insert

to enter this mode use the following after entering the normal mode

1. o (start editing Below the line where the cursor is)
2. i (start editing where the cursor is right now)
3. O (start editing Above the line where the cursor is)
4. I (start editing from the starting of the line)

Use Esc to exit a mode

now to execute any command come back to normal mode and press : to execute

Scripting

Bash script is just a series of commands

```
vim script.sh  
name='medhansh'  
number=12  
system=$(uname)  
  
#remember echo is just printf  
echo "hello $name (number is $number)"  
  
> output :→ hello medhansh (number is 12)
```

file permissions

- `ls -l` !:-> long listing
- now to change the permission of a file use `chmod`

now this command

```
chmod +x script.sh
# now +x means add executable permission

chmod 755 script.sh
# same as above
```

now here comes the main thing the one and only SHEBANG

```
#!/bin/bash
```

- This tells the system about what compiler should be the file executed from
- Or better one :- Which language i will be speaking for the rest of the file (English, dutch, esponal)

scripting

First Progame (Hello, World!)

```
vim hello
  echo "Hello, World!"
:wq

chmod +x hello
./hello
> Hello, World!
```

Variable expansion

```
vim expansion
  name='Medhansh'
  echo "Hello, $name!"
:wq

chmod +x expansion
./expansion
> Hello, Medhansh!
```

for loops

- for loops are something like iterating over a array
- something like javascript

```
vim for
  for var in med han sh jo sh; do
    echo "var is $var"
  done

chmod +x for
./for
> var is med
> var is han
```

```
> var is sh
> var is jo
> var is sh
```

syntax checker `bash -n`

used to check the syntax of your script

```
bash -n for
# no errors → nothing is displayed
# to check exit code of the last command use

echo $?

0          → worked like a charm
non-zero  → these is an issue
```

taking input

`read` is the command

```
vim input
  read -p "Enter your name: " name
  echo "Hello, $name"

chmod +x input
./input
> Enter your name : Medhansh
> Hello, Medhansh
```

- a trick `yes`
- `yes` is a valid command in your system use it when you want to say yes multiple times and you are aware of the steps

```
yes | ./input

> Hello, y
```

argc and argv

- count of the variables `argc` -> is stored in the variable `#`
- `argv` the array of the words -> is stored in the variable `@`

```
vim input
#!/usr/bin/env bash
# read -p "Enter your name: " name
echo "Hello, $1"

chmod +x input
./input Medhansh

> Hello, Medhansh
```

```
vim input
#!/usr/bin/env bash
if [[ -n $1 ]]; then
    name = $1 #if something is there in name || the name is not empty
else
    read -p "Enter your name: " name
    #if name comes out to be empty
fi #this is how to close the

echo "Hello, $1"
```

```
chmod +x input
```

```
.input Medhansh
> Hello, Medhansh
```

```
.input
> Enter your name: Medhansh
> Hello, Medhansh
```

```
help '['
help test
```

@ operator

```
vim for
for var in "$@"; do
    echo "var is $var"
done
```

```
chmod +x for
.for med han sh jo sh
> var is med
> var is han
> var is sh
> var is jo
> var is sh
```

```
.for med han sh
> var is med
> var is han
> var is sh
```

```
.for med han
> var is med
> var is han
```

```
.for med
> var is med
```

@ -> this is expand until there is no more left

Functions


```
hello(){
    name = $1
    # $1 → the first argument passed to the function
    #the name declared like this becomes a global variable
    so instead we use

}
```

```
hello(){
    local name = $1
    echo "Hello, $name"
    return 0
}
```

Conditionals

```
#!/usr/bin/env bash
a=2
b=2

if [[ $a == $b ]]; then
    echo "a and b are equal"
fi

c=2
d=3

if [[ $a != $b ]]; then
    echo "c and d are not equal"
fi

> a and b are equal
> c and d are not equal
```

checking in files **-f**

```
if [[ -f file.txt ]]; then
    echo 'file.txt is a file and it exists'
fi

> 'file.txt is a file and it exists'
```

TO get more about the flags use **help test**

a bit of **while** loop

```
while [[ -f file.txt ]]; do
    echo 'File.txt exists and is a file'
    sleep 1
done
## till file exists it will print file.txt exists and is a file
## sleep 1 → delay(1)
```

a bit about **until** loops

- until is opposite of while loops
- rarely used

```
until [[ -f file.txt ]]; do
    echo 'File.txt does exists'
    sleep 1
done
## untill file doesn't exist it will print 'file.txt doesn't exist'
## sleep 1 → delay(1)
```

using with commands

```
if ls; then
    echo 'ls worked'
fi

> all the files in the dir
> ls worked
```

true & false

```
if true ; then
    echo "it is true"
else
    echo "it is false"
fi
```

BackThoughts

```
if apt-get update; then
    echo 'system updated'
else
    echo 'not able to update the system'
fi
```

For loops

```
#!/usr/bin/env bash

for name in med han sh; do
    echo "name is $name"
done
```

{ } notation

- {} → Range in python

```
#!/usr/bin/env bash

for thing in {a..f}; do
    echo "this is $thing"
done

>this is a
```

```
>this is b
>this is c
>this is d
>this is e
>this is f
```

```
#!/usr/bin/env bash
```

```
for thing in {1..6}; do
    echo "this is $thing"
done
```

```
>this is 1
>this is 2
>this is 3
>this is 4
>this is 5
>this is 6
```

C like for loops

```
max=5
for((i =0; i<max;i++)); do
    echo "thing is $i"
done
```

```
>this is 0
>this is 1
>this is 2
>this is 3
>this is 4
```

input and output (I/O)

```
#!/usr/bin/env bash
read fo
echo "you said $fo"
```

this script above can cause errors
this won't keep data raw

```
You entered : 'hello \n world'
displayed : 'hello n world'
```

-r flag

so instead use this flag -r (raw)

```
#!/usr/bin/env bash
read -r fo
echo "you said $fo"
```

- line by line reading in bash is slow

regex

general expressions

for example

emails :- are like name@domain

now `name` and `domain` are variable here

now

`a*` -> a word that has a starting character a -> `'a'kriti`

`m*` -> a word that has a starting character m -> `'m'edhansh`

```
#name="medhansh"
read -pr "enter you name" name
if [[ $name = m* ]]; then
    echo "you are the great, $name"
elif [[ $name = j* ]]; then
    echo "you are not as great as M, $name"
else
    echo "No need of yours"
fi

> enter your name : medhansh
> you are the great, medhansh

> enter your name : jayant
> you are not as great as M, jayant

> enter your name : rehan
> No need you yours
```

case statements

```
s = $1

case "$s" in
    medhansh)
        echo hi medhansh
        ;;
    jayant)
        echo hi jayant
        ;;
    rehan)
        echo so ja
        ;;
    *)
        echo hello world
esac

> ./case medhansh
> hi medhansh

> ./case jayant
hi jayant
```

- `;;` -> this acts like a `break`;
- `*` -> default case
- `;&` -> this is for when you want default behaviour just like in C language

Classic C behaviour

```
s='medhansh'

case "$s" in
    m*) echo 'matched m*';&
    medhansh) echo 'matched medhansh';&
    f*) echo 'matched f*';&
    cati) echo 'matched cati';&
    *) echo 'matched *';&
esac

> matched m*
> matched medhansh
> matched f*
> matched cati
> matched *
```

Typical Behaviour

```
s='medhansh'

case "$s" in
    m*) echo 'matched m*';;
    medhansh) echo 'matched medhansh';;
    f*) echo 'matched f*';;
    cati) echo 'matched cati';;
    *) echo 'matched *';;
esac

> matched m*
```

New kind of switch-case

`;;&` -> checks every case if matches then does the action

```
s='medhansh'

case "$s" in
    m*) echo 'matched m*';;&
    medhansh) echo 'matched medhansh';;&
    f*) echo 'matched f*';;&
    cati) echo 'matched cati';;&
    *) echo 'matched *';;&
esac

> matched m*
> matched medhansh
> matched *
```

Indexed-Array

```

array=(med han sh jo)

echo "${array[0]}"
echo "${array[1]}"
echo "${array[2]}"
echo "${array[3]}"
echo "${array[4]}"

> med
> han
> sh
> jo
>
#bash will treat it like an empty

```

-ve indices

```

array=(med han sh jo)

echo "${array[-1]}"
echo "${array[-2]}"
echo "${array[-3]}"
echo "${array[-4]}"

> jo
> sh
> han
> med

```

indices using variable

```

idx=2

array=(med han sh jo)

echo "${array[$idx]}"
echo "${array[idx]}"
#these both syntax are valid in bash

```

@ && * in bash

```

array=(med han sh jo)

for i in "${array[*]}"; do
    echo "item is $i"
done

> med han sh jo

```

```

array=(med han sh jo)

for i in "${array[@]}"; do
    echo "item is $i"
done

```

```
> med
> han
> sh
> jo
```

- `*` -> string-ifies the array (aka converts array into the string)
- `@` -> treat it like a normal array

copying an array and add elements



```
first_array=(
  med
  han
  sh
  jo
)

second_array=("${first_array[@]}")

second_array+=(sh i)

for i in "${second_array[@]}"; do
  echo "item is : $i"
done
```

- `echo $array` gives the first element of the array

Another way to declare array (Sparse arrays)

```
array=([0]="med" [1]="han" [2]="sh")

#why is this used

array=([0]="med" [1]="han" [2]="sh" [23]="jo")
```

- get the length of the array
`echo "${#array[@]}"`
- Get the length of the string
`echo "${#array[23]}"` -> length of the word at 23rd index

Associative Array (These are relatively new)

- Something like hash Map

- We need to use declare them using `declare`

```
#!/usr/bin/env bash
```

```
declare -A arr
arr[med]=11
arr[han]=23
arr[sh]=9

echo "${arr[med]}"

> 11
```

! -> in-direction operator

getting a value in Associative array

- Spoiler alert we will use `!`

```
#!/usr/bin/env bash
```

```
declare -A arr
arr[med]=11
arr[han]=23
arr[sh]=9

echo "${!arr[*]}"

> med han sh
#remember * operator will stringi-fies
```

IFS variable

- built-in to bash
- it tell how to stringi-fy the array (from which character)

```
array=(med han sh)

echo "array is ${array[*]}"

> med han sh
```

- unset variables using `unset variable-name`

```
array=(med han sh)

IFS=,
echo "array is ${array[*]}"

> med,han,sh
```

Command Substitution

- running a command in bash and storing its output
- we need to use backticks

```
thing=`whoami`  
echo thing
```

- nesting them becomes a problem

better way to substitute the commands (dollar-pram)

```
thing="$(whoami)"  
echo "$thing → $(uname)"
```

```
> medhansh → linux
```

-> using these they run commands in a different subshell

```
i=5  
funct(){  
    echo "i am $1"  
    i=6  
}  
  
echo "$(funct hello)"  
echo "i is $i"  
  
> i am hello  
> i is 5  
# all the change it made inside the function is dropped once the function is over
```

- these notation can't modify the global scope

another syntax for the substitution (GLOBAL one)

- introduced in 2025 bash 5.3

```
i=5  
funct(){  
    echo "i am $1"  
    i=6  
}  
  
echo "${ funct hello; }"  
echo "i is $i"  
  
> i am hello  
> i is 6
```

- this will change the variable
- it didn't run in its own subshell
- hence making it fast as no new subshell was made

Arithmetic Expressions

```
thing=$(( 12+34 ))  
echo $thing
```

```
> 46
```

- `**` -> exponent
- `*`
- `/`
- `-`
- `+`

```
q=1  
w=2  
  
echo $(( $q + $w ))
```

```
q=1  
w=2  
  
echo $(( q + w ))
```

Some more

- `++i`
- `i++`
- `i *= 3`
- `| |` << left shit and right shift
- ternary

using ternary

```
a=12  
b=2  
((max = a > b ? a:b))  
  
echo $max
```

((....)) this thing has a return code too
- 0 (if passed or the answer is something)
- 1 (failed or the answer was 0)

```
a=05  
echo $a  
echo $(( a ))  
  
> 05  
> 5
```

```
a=06
echo $a
echo $(( a ))

> 06
> 6
```

```
a=07
echo $a
echo $(( a ))

> 07
> 7
```

```
a=08
echo $a
echo $(( a ))

> error
```

- using zero in-front bash treats it like an octal number (base 8)
- we need to be careful when using it

Forcing into a specific base

- using `#`

```
a=08
echo $a
echo $(( 10#$a ))

> 08
> 8
```

Process substitution

- to get the count of the all the words

```
grep -c d /usr/share/dict
```

```
#!/usr/bin/env bash
words=$(grep d /usr/share/dict/words)
I=0
for words in $words;do
    echo $word
    ((I++))
done

echo "count is : $I"
```

- now the issue with this code it that it is waiting for command to run and then do the processing
- what if the words were 90M or more

Here Strings

```
#!/usr/bin/env bash
words=$(grep d /usr/share/dict/words)
I=0
while read -r words;do
    echo $word
    ((I++))
done <<< "$words"

echo "count is : $I"
```

- same issue as above
- first is processing and then comes out thing
- what about doing this in a streaming fashion

Another type of syntax to go with

```
#!/usr/bin/env bash
I=0
grep d /usr/share/dict/words | while read -r words;do
    echo $word
    ((I++))
done

echo "count is : $I"
```

- the answer will be correct because the command right to the pipe runs in a sub-shell

Finally command substitution

- remember about the `<` this was used way back to read and write from a file (mostly read for writing the other one is used)

```
#!/usr/bin/env bash
I=0
while read -r words;do
    echo $word
    ((I++))
done< <(grep d /usr/share/dict/words)

echo "count is : $I"
```

- now the while loop and the finding work on the same time hence increasing your script speed when working on something important
- the issue with this is
 - what is the return code (how do i get to know the command ran as usual)
 - the answer is : you don't you need to be sure about what you are trying to access (validate the input/data before processing)
 - in short (technical) it will mask the exit codes of the commands so be AWARE
- dissecting the script even further
 - now see this syntax `done< <(.....)`
 - now remember the first line i wrote before starting this session
 - this grep command has to return in some sort of file for processing

- *so when ever trying to use in future you need to make sure the command does return a file*
- some more examples for better understanding
- `echo $(uname)` we discussed this and it gave the name of your system
- `echo <(uname) ->` from the above logic it should return something file type
 - it returns a file descriptor
- `cat <(uname)` -> remember cat was used to read files
 - it will give the output you were hoping for

SUMMARY HERE

TREATING IT LIKE A FILE

cut && tr

```
echo $PAST
```

```
/home/medhansh/.local/bin:/usr/bin:/usr/local/sbin:/usr/local/bin:/usr/bin:/var/lib/flatpak/exports/bin:/usr/bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl:/home/medhansh/.local/bin:/home/medhansh/.local/bin
```

now we'll add the operations on this

```
echo $PAST | tr : '\n'
```

-----OUTPUT-----

```
/home/medhansh/.local/bin
/usr/bin
/usr/local/sbin
/usr/local/bin
/usr/bin
/var/lib/flatpak/exports/bin
/usr/bin/site_perl
/usr/bin/vendor_perl
/usr/bin/core_perl
/home/medhansh/.local/bin
/home/medhansh/.local/bin
```

```
cat hello.csv | cut -d , 1-
```

- -d tells the cut - were to break it from
- 1- means till the end of the columns

sed , awk && grep

The Hunter (GREP)

```
grep "medhansh" name.txt
```

-i flag

```
grep -i "medhansh" name.txt
```

- ignores the case `medhansh` `Medhansh` `medhAnsh` `medHansh`
- all the names are matched

-r flag

```
grep -r "medhansh" .
```

- search i the current directory
- don't stop after searching one file

-n flag

```
grep -n "medhansh" name.txt
```

- show the line numbers too

-v flag

```
grep -v "medhansh" name.txt
```

- exclude medhansh from the result
- or work opposite of the grep

-c flag

```
grep -c "medhansh" name.txt
```

- number of times medhansh has been mentioned in name.txt

regex

start

```
grep "^medhansh" name.txt
```

- all the line starting with medhansh
(i know all the lines 🤔)

end

```
grep "medhansh$" name.txt
```

- all the lines ending with medhansh

lines

```
grep "[0-9]" name.txt
```

- all the lines containing numbers

SED (stream editor)

-transforms into something

FILE rn

```
medhansh
rehan
neeraj
```

```
sed 's/medh/ansh' name.txt
```

```
ansh
rehan
neeraj
```

-i flag

- inplace
- make the change in a file rather than a stream

```
sed -i 's/medh/ansh/g' name.txt
```

delete a line

```
sed '3d' name.txt
```

- delete line number 3

```
sed '/rehan/d' name.txt
```

- deletes all the lines with the word rehan in them

replacement

```
sed '2s/apple/orange/' file.txt
```

- replace the secon line

awk

- it the one of the extensive and the useful tool
- a new language in itself

print a specific column

```
awk '{print $1}' name.txt
```

```
awk '{print $1 $3}' name.txt
```

the one and only the BEAST the POWER the INSPIRATION printf

```
awk '{printf("first name is %s, and the last is %s", $1,$2)}' name.txt
```

change the delimiter `-F`

- same thing as IFS but opposite for work
- IFS were for joining based on it and
- delimited is the character to find where to start breaking from

```
awk -F "," '{print $1}' name.txt
```

conditionals

```
awk '$3 > 50 {print $1}' marks.txt
```

built-in variables

variable name	notation
\$0	Entire line
\$1	First column
NF	Number of fields
NR	Line Number

Sort and Uniq

sort

- sorts by-default on the Alphabets
- piping is the easiest option

```
stream | sort
```

uniq

- the string/stream should be sorted first hand to use uniq

using combined

```
stream | sort | uniq -c
```

- -c for the count
- it makes a basic frequency counter

Word Count (wc)

- useful for counting the raw bytes
- don't work for emojis 🙅

```
/usr/share/dict/words | grep m | wc
```

-l -> counts lines

-w -> count words

-c -> counts characters

Find

- useful for finding the files

```
find <starting-point> <conditions> <actions>
```

```
find . -name "*.txt"
```

- Start at `.`
- For each file
- If name matches `*.txt` → print it.
- printing is default action

-l flag

```
find /usr/share | wc -l  
#-l :→ to get the total number
```

-type flag

```
find /usr/share -type f | wc -l  
# f :→ to get the files only (PLAIN FILES)
```

```
find /usr/share -type d | wc -l  
# d :→ to get the directories only
```

```
find /usr/share -type l | wc -l  
# l :→ to get the symbolic links
```

regex

```
find /usr/share -type f -name '*.txt'
```

find with exec

```
find /usr/share -type f -name '*.txt' -exec echo file name : {} <- this is ';' <- this is /; #using escaping method
```

```
>file name : /usr/share/Z11/Xcms.txt <- this is
```

Bash (Bash as a command) arguments

- our script

```
#!/usr/bin/env bash

name1=kumar
name2=pankaj

full_name="$name1 $name2"

if [[first_name == 'Neeraj' ]]; then
    echo "Medhansh is the best"
fi

echo "Anyways Medhansh is best"
```

----OutPut-----

| Anyways Medhansh is best

__How TO run__

- `bash script` || (if you have give it executable permissions)
- `./script`

debug mode

```
bash -x script

+ name1=kumar
+ name2=pankaj
+ full_name='kumar pankaj'
+ [[ kumar == neeraj]]
+ echo 'Medhansh is great'
+ echo 'Anyways Medhansh is best'
```

another way for using debugging mode

```
#!/usr/bin/env bash

set -x
name1=kumar
name2=pankaj

full_name="$name1 $name2"

if [[first_name == 'Neeraj' ]]; then
    echo "Medhansh is the best"
fi

echo "Anyways Medhansh is best"
```

```
./script
```

```
+ name1=kumar
+ name2=pankaj
+ full_name='kumar pankaj'
+ [[ kumar == neeraj]]
+ echo 'Medhansh is great'
+ echo 'Anyways Medhansh is best'
```

Targeting certain part of the code

```
#!/usr/bin/env bash

set -x
name1=kumar
name2=pankaj

set +x

full_name="$name1 $name2"

if [[first_name == 'Neeraj' ]]; then
    echo "Medhansh is the best"
fi

echo "Anyways Medhansh is best"
```

```
./script

+ name1=kumar
+ name2=pankaj
+ set +x
+ echo 'Medhansh is great'
+ echo 'Anyways Medhansh is best'
```

PS4 - variable

```
PS4='hello : ' bash -x script

hello : name1=kumar
hello : name2=pankaj
hello : full_name='kumar pankaj'
hello : [[ kumar == neeraj]]
hello : echo 'Medhansh is great'
echo 'Anyways Medhansh is best'
```

- using `-x` for debug

```
PS4='[debug]: ' bash -x script

[debug]: name1=kumar
[debug]: name2=pankaj
[debug]: full_name='kumar pankaj'
[debug]: [[ kumar == neeraj]]
[debug]: echo 'Medhansh is great'
echo 'Anyways Medhansh is best'
```

using `-n` flag

- it doesn't runs the code actually

```
bash -n ./script

# to find the syntax error
```

- return some values
- returns `0` if write
- returns some kind of +ve value when there is a error

using `-u` flag

- to find if there is an undefined variable
- by default variable have global scope (the same that these variables have `{$SHELL $PATH $- $whoami ...}`)
- so it could lead to some catastrophes like overwriting some built-in variable

```
bash -u script
```

Timing commands

- `time command`
- `time ls`

time	usage
real time	time that you will feel
user time	time spent in user space
sys time	time spent in the system or the kernel

- time is both built-in into the bash and is also an external command

Sourcing code

- it is something like `importing` or `including`

```
#!/usr/bin/env bash

source ./lib/

p medhansh
p jayant
```

another way of sourcing `.`

```
#!/usr/bin/env bash

. ./lib/
```

```
p medhansh
p jayant
```

```
#!/usr/bin/env bash
```

```
. ./lib/ || exit
```

```
p medhansh
p jayant
```

- something like short-circuiting (JS)

```
if __name__ == '__main__':
```

```
if ! (return 2>/dev/null); then
    # do some stuff
fi
```

- if we are able to return then we are in the main file and if we are not then we are no in the file(we are being sourced)

{curls} and (paranthesis)

- let's talk about an issue first
the issue:- see the following code
- script-Name := functions

```
#!/usr/bin/env bash
```

```
greet(){
    name=$1
    echo "how r u $name"
}
```

```
bye(){
    name=$1
    echo "bye $name"
}
```

- let's source this file

```
#!/usr/bin/env bash
```

```
. ./lib/functions
name=rehan
```

```
echo "before $name"
greet medhansh
echo "after $name"
bye medhansh
```

```
> before rehan
> how r u medhansh
```

```
> after medhansh
> bye medhansh
```

- look closely why did the name from the current script changed
reason is pretty simple variables have global scope

so there are two fixes for this problem

1. either add the `local` keyword before variable inside the function
2. or use subshell (remember from the previous knowledge we know anything inside or any change inside a subshell is destroyed once it ends :-like you don't remember what was taught in the class once it finishes)

Fix-1

change the main script like the following

```
#!/usr/bin/env bash

greet(){
    local name=$1
    echo "how r u $name"
}

bye(){
    local name=$1
    echo "bye $name"
}
```

```
#!/usr/bin/env bash

. ./lib/functions
name=rehan

echo "before $name"
greet medhansh
echo "after $name"
bye medhansh

> before rehan
> how r u medhansh
> after rehan
> bye medhansh
```

everything is fixed and works as intended

Fix-2 (Using subshell)

```
#!/usr/bin/env bash

greet(){
    name=$1
    echo "how r u $name"
}
```

```
bye(){
    name=$1
    echo "bye $name"
}
```

```
#!/usr/bin/env bash
```

```
. ./lib/functions
name=rehan

echo "before $name"
greet medhansh
echo "after $name"
bye medhansh
```

```
> before rehan
> how r u medhansh
> after rehan
> bye medhansh
```



now when using this kind of this :- we need to add ;

```
#!/usr/bin/env bash
```

```
. ./lib/functions
name=rehan

echo "before $name"
greet medhansh
{echo "after $name"; }
bye medhansh
```

```
> before rehan
> how r u medhansh
> after rehan
> bye medhansh
```

RETURN AND OUTPUT

```
#!/usr/bin/env bash
```

```
my-func(){
    echo 'this goes to stdout'
    return 0
}
var=$(my-func)
code=$?

echo "output=$var, code=$code"
```

1

- return codes are 8-bit integers (0-255)

```
#!/usr/bin/env bash
```

```
my-func(){  
    echo 'this goes to stdout'  
    return 256  
}  
var=$(my-func)  
code=$?  
  
echo "output=$var, code=$code"
```

```
output=this goes to stdout, code=0
```

2

```
var=$(my-func)
```

var only catches the standard output

```
#!/usr/bin/env bash
```

```
my-func(){  
    echo 'this goes to stdout' >&1  
    echo 'this goes to stderr' >&2  
    return 0  
}  
var=$(my-func)  
code=$?  
  
echo "output=$var, code=$code"
```

```
this goes to stderr
```

```
output=this goes to stdout, code=0
```

redirecting the outputs

```
#!/usr/bin/env bash
```

```
my-func(){  
    echo 'this goes to stdout' >&1  
    echo 'this goes to stderr' >&2  
    return 0  
}  
var=$(my-func >/dev/null)  
code=$?  
  
echo "output=$var, code=$code"
```

```
this goes to stderr
```

```
output= , code=0
```

- throws all standard the output

throwing specific type output

```
#!/usr/bin/env bash

my-func(){
    echo 'this goes to stdout' >&1
    echo 'this goes to stderr' >&2
    return 0
}
var=$(my-func 1>/dev/null)
code=$?

echo "output=$var, code=$code"
```

```
this goes to stderr
output= , code=0
```

- this is the same as throwing everything to `/dev/null` but explicitly stating it (`1`)

```
#!/usr/bin/env bash

my-func(){
    echo 'this goes to stdout' >&1
    echo 'this goes to stderr' >&2
    return 0
}
var=$(my-func 2>/dev/null)
code=$?

echo "output=$var, code=$code"
```

```
output=this goes to stdout, code=0
```

- all the stderr goes to `/dev/null`

```
#!/usr/bin/env bash

my-func(){
    echo 'this goes to stdout' >&1
    echo 'this goes to stderr' >&2
    return 0
}
var=$(my-func 2>&1)
code=$?

echo "output=$var, code=$code"
```

```
output=this goes to stdout
this goes to stderr, code=0
```

- redirecting the stderr to stdout

Parameter Expansion

Bash's built-in mechanism to

retrieve,
modify,
test,
transform

the value of a variable — *without* calling external tools like `sed`, `awk`, or `cut`.

it is fast because it runs inside the shell itself.

basic syntax:

```
${parameter}
```

Why use Parameter Expansion?

- avoid spawning external processes
 - improve performance in scripts
 - perform string manipulation directly
 - handle defaults, errors, substring extraction, pattern removal.
-

Simple Expansion

```
name="Medhansh"  
echo ${name}
```

Output:

```
Medhansh
```

Braces vs No Braces

```
name=medhansh  
echo $name123      # looks for variable "name123"  
echo ${name}123    # expands "name" and appends 123
```

output:-

```
nothing will be here because name123 doesn't exist  
medhansh123
```

Use Default if Variable is Unset or Null :-

```
echo ${var:-"default_value"}
```

- If `var` is unset or empty → prints `default_value` (something like `function(int name="Medhansh")`)
-

Assign Default Value if Unset `:=`

```
echo ${var:= "default_value"}
```

- This assigns `"default_value"` to `var` if it was unset.
-

Use Alternate Value if Set `:+`

```
echo ${var:+ "alternate"}
```

- If `var` exists → prints `"alternate"`
-

Display Error if Unset

```
echo ${var:? "Variable not set"}
```

- Stops script execution if `var` is unset.
-

String Length

```
echo ${#var}
```

- Returns number of characters.
-

Substring Extraction

Syntax:

```
${var:start:length}
```

Example:

```
text="HelloWorld"  
echo ${text:0:5}
```

Output:

```
Hello
```

From Specific Position to End

```
echo ${text:5}
```

Output:

```
World
```

Remove Pattern from Start

Remove Shortest Match

```
file="backup.tar.gz"  
echo ${file#*.}
```

Output:

```
tar.gz
```

Remove Longest Match

```
echo ${file##*.}
```

Output:

```
gz
```

Remove Pattern from End

Remove Shortest Match

```
echo ${file%.*}
```

Output:

```
backup.tar
```

Remove Longest Match

```
echo ${file%%.*}
```

Output:

```
backup
```

Replace String

Replace First Match

```
echo ${var/pattern/replacement}
```

Replace All Matches

```
echo ${var//pattern/replacement}
```

Example:

```
text="apple banana apple"  
echo ${text//apple/orange}
```

Output:

```
orange banana orange
```

Change Case

Convert to Uppercase

```
echo ${var^^}
```

Convert to Lowercase

```
echo ${var,,}
```

Capitalize First Letter

```
echo ${var^}
```

Remove Matching Pattern

Remove Prefix Pattern

```
echo ${var#pattern}
```

Remove Suffix Pattern

```
echo ${var%pattern}
```

Indirect Expansion

```
name="var"  
var="Hello"  
echo ${!name}
```

Output:

```
Hello
```

Array Expansion

All Elements

```
echo ${array[@]}
```

Number of Elements

```
echo ${#array[@]}
```

Practical Example

Extract filename without extension:

```
file="document.pdf"
echo ${file%.*}
```

Output:

```
document
```

Basic Globbing

```
ls files/
```

```
> med.txt
> han.txt
> sh.txt
> med.md
> han.md
> sh.md
```

```
echo files/*
# echo everything inside the files directory
> files/ans.md files/ans.txt files/med.md files/med.txt files/sh.md files/sh.txt

# BTS
# bash actually ran this command 🙌
echo files/ans.md files/ans.txt files/med.md files/med.txt files/sh.md files/sh.txt
```

```
printf '%s\n' files/*
# %s will be repeated again and again
```

```
> files/ans.md
> files/ans.txt
> files/med.md
> files/med.txt
> files/sh.md
> files/sh.txt
```

```
ls -1 files/m[edh]a.*
# all the files with either e,d,h at the second position of the name and anything at the end
> files/med.txt
> files/med.md
```

symbol	meaning
*	Give me anything... any length... even nothing.
?	Give me exactly one mystery character.

Extended Globbing

Shell options

shopt -> some options built into the bash

-s to set something on

-u to unset something to off

```
# help → built in command
help shopt
```

```
# lists all the options
shopt
```

```
# sets it on
shopt -s extglob
```

```
# sets it off
shopt -u extglob
```

negating something

```
ls -1 files/*.txt
# gets you all the txt files
```

```
ls -1 files/!(*.txt)
# gets you everything except .txt files
```

```
ls -1 files/+(med|han).txt
#match med or han at-least on or more time

# for example
> medmed.txt
> hanhan.txt
> med.txt
> han.txt
```

symbol		defenation
+(sth)	=>	1 ore many times
?(sth)	=>	0 or 1 time
*(sth)	=>	0 or many times


```
ls -l files/@med.txt
#match med exactly

# for example
> med.txt
> med.md
```

More GLOB options

```
[medhansh@archlinux ~]$ shopt | grep glob
dotglob          off
extglob          off
failglob         off
globasciiranges  on
globskipdots     on
globstar         off
nocaseglob       off
nullglob         off
```

extglob

failglob

globasciiranges

globskipdots

nocaseglob

nullglob

```
mkdir empty
```

```
ls empty/
>
```

```
ls empty/*
> ls: empty/*: No such file or directory
```

Is got literal string `empty/*`

- to prove this

```
print "<%s>" empty/*

> <empty/*>
# it actually got literal `empty/*`
```

- so lets fix this in following way

```
touch empty/hello.txt
```

- lets try it again

```
print "<%s>" empty/*
```

```
> <empty/hello.txt>
```

- works as it was intended
- now let's achive the same behaviour in better manner

```
shopt -s nullglob
```

- let's retry

```
printf '<%s>\n' empty/*
```

```
> ◇
```

dotglob

```
touch empty/.hiddenFile
```

```
echo empty/*
```

```
> empty/*
```

- although the file exists in the directory yet it doesn't shows the file(it is hidden file)

```
shopt -s dotglob
```

- now this will show the dotfiles too

```
echo empty/*
```

```
> empty/.hiddenFile
```

globstar

- let's try to get the same behaviour as find .

```
find .
```

- OUTPUT

```
.
./metadata.json
./contents
./contents/defaults
./contents/previews
./contents/previews/preview.png
.> /contents/previews/splash.png
./contents/previews/fullscreenpreview.jpg
./contents/splash
```

```
./contents/splash/images
./contents/splash/images/busy02.svg
./contents/splash/images/background.png
./contents/splash/images/logo.png
./contents/splash/Splash.qml
```

```
shopt -s globstar
```

```
printf '%s\n' **
```

- OUTPUT

```
.
./metadata.json
./contents
./contents/defaults
./contents/previews
./contents/previews/preview.png
.> /contents/previews/splash.png
./contents/previews/fullscreenpreview.jpg
./contents/splash
./contents/splash/images
./contents/splash/images/busy02.svg
./contents/splash/images/background.png
./contents/splash/images/logo.png
./contents/splash/Splash.qml
```

some cool things

```
txt=(./**/*.txt)

echo "${txt[@]}"
```

brace expansion

```
echo {med,han,sh,jo}
> med han sh jo

printf '%s\n' {med,han,sh,jo}
> med
> han
> sh
> jo
```

another use case

```
filename='myFile'

echo "${filename}.{txt,md,jpg}
```

```
> myFile.txt
> myFile.md
> myFile.jpg
```

```
arr=(/etc/{med,han,sh}/{1,2,3}.{txt,md,jpg})

for item in "${arr[@]};do
    echo "$item"
done
```

---OUTPUT---

```
/etc/med/1.txt
/etc/med/1.md
/etc/med/1.jpg
/etc/med/2.txt
/etc/med/2.md
/etc/med/2.jpg
/etc/med/3.txt
/etc/med/3.md
/etc/med/3.jpg
/etc/han/1.txt
/etc/han/1.md
/etc/han/1.jpg
/etc/han/2.txt
/etc/han/2.md
/etc/han/2.jpg
/etc/han/3.txt
/etc/han/3.md
/etc/han/3.jpg
/etc/sh/1.txt
/etc/sh/1.md
/etc/sh/1.jpg
/etc/sh/2.txt
/etc/sh/2.md
/etc/sh/2.jpg
/etc/sh/3.txt
/etc/sh/3.md
/etc/sh/3.jpg
```

Brace and Globbing

```
touch {med,han,sh}.{jpg,txt}
```

- another way

```
ls files/*.{txt,md}
```

```
# any file with extension .txt or .md
```

Numeric Brace Expansion

```
echo {1,2}
```

```
> 1 2
```

```
echo {1..2}
```

```
> 1 2
```

```
echo {1..20}
```

```
> 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

```
echo {10,5}
```

```
> 10 9 8 7 6 5
```

```
echo {1..100..5} # 1 to 100 skipping by 5
```

```
> 1 6 11 16 21 26 31 36 41 46 51 56 61 66 71 76 81 86 91 96
```

seq command

```
seq 1 10
```

```
1
2
3
4
5
6
7
8
9
10
```

- we don't need seq command
- we can do the same using bash

the ★ of the show `printf`

- printing a formatted data

What is printf?

`printf` is a Bash builtin command used to print formatted output to the terminal.

It gives you precise control over how text and numbers appear — including spacing, alignment, decimals, tabs, and newlines.

Unlike `echo`, it behaves consistently across different systems and shells.

Basic Syntax

```
printf "format_string" arguments
```

- format_string → template that contains placeholders
 - arguments → values that replace placeholders
-

Format Specifiers

Placeholders used inside the format string:

Specifier	Meaning
%s	String
%d	Integer
%f	Float
%c	Character
%%	Literal % symbol

Example:

```
printf "Age: %d\n" 18
```

Output:

```
Age: 18
```

Escape Sequences

Used to control formatting of output:

Escape	Meaning
\n	New line
\t	Tab space
\	Backslash
"	Double quote
\b	Backspace

Example:

```
printf "Name:\tJohn\nAge:\t18\n"
```

Width and Alignment

Right-aligned:

```
printf "%5d\n" 42
```

Output:

```
42
```

Left-aligned:

```
printf "%-5d\n" 42
```

Output:

```
42
```

Floating Point Precision

```
printf "%.2f\n" 3.14159
```

Output:

```
3.14
```

Multiple Arguments

```
printf "%s scored %d marks\n" "Raj" 89
```

Printf replaces placeholders in order.

Assign Output to Variable

Use `-v` flag to store formatted output into a variable instead of printing:

```
printf -v msg "Hello %s" "Medhansh"
echo "$msg"
```

Output:

```
Hello Medhansh
```

Why printf is Better than echo

- Consistent across all shells
- Supports formatting
- Controls decimal precision
- Allows alignment and spacing
- Supports variable assignment

Recommended for scripting where output formatting matters.

Practical Use Case

Creating formatted tables in terminal:

```
printf "%-10s %5d\n" "User" 1200
printf "%-10s %5d\n" "Admin" 950
```

Output:

```
User      1200
Admin     950
```

DATE FORMATING

```
date
```

- get the current time

```
datefmt="%Y/%m/%d %H:%M:%S"
date +"$datefmt"
> 2026/02/25 02:41:37
```

- ISSUE:- all the versions of date don't behave consistently hence the greatest `printf` had to come

```
printf 'the date is %T(%Y/%m/%d %H:%M:%S)\n'
```

- issue with this lets say we need to add something else after time


```
printf 'the date is %T(%Y/%m/%d %H:%M:%S)\n name is %s' medhansh
```

this will lead to error

- so we do this instead

```
printf 'the date is %T(%Y/%m/%d %H:%M:%S)\n name is %s' -1 medhansh
```

- now this works as intended
- to find number of seconds this shell has been running

```
echo $SECONDS
```

- to get epochtime
(THE NUMBER OF SECONDS THAT HAVE PASSED SINCE 1 JANUARY 1970)

```
echo $EPOCHTIME
```

Regular Expressions

i need to work on it

mapfile

- better way to read data from a file
- instead of doing this

```
lines=()
while IFS= read -r line; do
    lines+=("$line")
done < file.txt
```

- we can just use the following

```
mapfile lines < file.txt

echo "${lines[0]}"
echo "${lines[1]}"
```

- line of the file = one array element.
- It reads input line-by-line from:

1. a file
2. a pipe
3. command output

```
mapfile files < <(ls)
```

captured each file as a array element

use of `-t`

- Now lines are clean.

```
mapfile -t files < <(ls)
```

- This is faster and cleaner than `while read`.

Bracket vs Test

`[[` vs `[` vs `test`

- when you are using single square bracket u need to quote all your expansion
- we don't need to quote in `[[`

```
#!/usr/bin/env bash
```

```
if test -f med.md; then
    echo med exists
fi
```

- simple programme checks for `med.md` and echo's if exists

```
#!/usr/bin/env bash
```

```
if [-f med.md]; then
    echo med exists
fi
```

- this works too

```
#!/usr/bin/env bash
```

```
if [[-f med.md]]; then
    echo Med exists
fi
```

- this works too

`[[` does everything that `[` and `test` command does

- `[[` these support the pattern matching too

```
#!/usr/bin/env bash
```

```
file='text.txt'
```

```
if [[ $file = *.txt ]]; then
    echo "$file is a txt file"
fi
```

`[` these don't support the expansion and wild-card behaviour

- inside `[[` the `' '` strings can be used to disable the expansion

- working with numbers is bit ugly with them

```
#!/usr/bin/env bash
a=9
b=10

if [[ $b > $a ]]; then
    echo "$b is greater than $a"
else
    echo this is an issue
fi
```

this is an issue

- this behaviour can be explained as following (a and b are compared as if they were strings)
- so using this causes errors

```
#!/usr/bin/env bash
a=9
b=10

if [[ $b -gt $a ]]; then
    echo "$b is greater than $a"
else
    echo this is an issue
fi
```

- this works as intended
- or better we can use ((

```
#!/usr/bin/env bash
a=9
b=10

if (( $b > $a )); then
    echo "$b is greater than $a"
else
    echo this is an issue
fi
```

- this works as expected:w

Special Strings

```
echo 'hello $USER'
```

hello \$USER

```
echo "hello $user"
```

hello medhansh

```
echo "hello\t$USER"
```

```
hello\tmedhansh
```

```
echo -e "hello\t$USER"
```

```
hello medhansh
```

```
echo '$hello\t$USER'
```

```
hello medhansh
```

```
\v -> vertical tab \n\t
```

Trap Signals

- signals are handled by the os

```
bash
[medhansh@archlinux ~]$ trap -l
1) SIGHUP  2) SIGINT  3) SIGQUIT  4) SIGILL  5) SIGTRAP
6) SIGABRT 7) SIGBUS  8) SIGFPE  9) SIGKILL 10) SIGUSR1
11) SIGSEGV 12) SIGUSR2 13) SIGPIPE 14) SIGALRM 15) SIGTERM
16) SIGSTKFLT 17) SIGCHLD 18) SIGCONT 19) SIGSTOP 20) SIGTSTP
21) SIGTTIN 22) SIGTTOU 23) SIGURG 24) SIGXCPU 25) SIGXFSZ
26) SIGVTALRM 27) SIGPROF 28) SIGWINCH 29) SIGIO 30) SIGPWR
31) SIGSYS 34) SIGRTMIN 35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

- all the signals we can trap
- this SIGINT is the signal sent when we press `ctrl+c`

```
#!/usr/bin/env bash
```

```
cleanup(){
    echo cleanup function running
}
trap cleanup exit

echo script starting
echo ...
echo script done
```

----OUTPUT-----

```
script starting
```

```
...
```

```
script done
```

```
cleanup function running
```

- a bit of surgery
`trap cleanup exit` this means when the script ends/quits this will be called

```
#!/usr/bin/env bash
```

```
cleanup(){  
    echo cleanup function running  
}  
trap cleanup exit  
  
echo script starting  
echo ...  
echo script done  
exit 1
```

```
script starting  
...  
script done  
cleanup function running
```

```
echo $?
```

```
1
```

- the script will exit with the code mentioned unless you override it

```
#!/usr/bin/env bash
```

```
cleanup(){  
    echo cleanup function running  
    exit 2  
}  
trap cleanup exit  
  
echo script starting  
echo ...  
echo script done  
exit 1
```

```
script starting  
...  
script done  
cleanup function running
```

```
echo $?
```

```
2
```

Named Pipes

- like a c program can communicate with the bash programme
- Named Pipes — also called FIFOs (First In First Out).
- A named pipe is a special type of file that allows two different processes to talk to each other.

Process A → Named Pipe → Process B

Why named pipes?

```
ls | grep txt
```

Normal pipes only work between processes started together in the same command.

Those processes must:

- share the same parent shell
- start at the same time

They're basically forced into the same arranged marriage.

named pipes remove these limitations

Creating a named pipe

```
mkfifo pipe
```

```
ls -l mypipe
prw-r--r-- 1 medhansh users 0 Feb 26 01:00 mypipe
```

- this `p` at the staring, tells us it is not a file .This is a FIFO pipe pretending to be one

```
# in terminal -1
cat > pipe
```

```
## this process is waiting something to write into it
```

```
# in terminal -2
cat < pipe
```

```
## this process is writing to it
```

we type in the terminal 1 and the text is instantly teleported to the terminal -2
on file nor nothing

example (CHATGPT)

Where This Gets Real

Now imagine:

1. A Python program logs data
 2. A Bash script processes it
 3. A C program visualizes it
- All in real-time
 - All separately running

- All communicating through a FIFO

color Outputs

- ANSI Escape Sequences
- (AMERICAN NATIONAL STANDARDS INSTITUTE)✅
- (AMERICAN INDIAN)❌
- general idea

```
<some tag> S0me thign here <closing tag>
```

basic structure (a bit ugly)

```
\033[STYLE;COLORMTEXT\033[0m
```

breakdown

symbol	usage
\033 →	escape character (ESC). This tells the terminal: control sequence incoming.
[→	begins the control command
STYLE;COLOR →	instructions
m →	apply formatting
TEXT →	your visible output
\033[0m →	reset everything (this is important or your whole terminal becomes radioactive)

echo -e flag

- sometimes `echo` would betray you
- so you need to explictly tell it to interpret the ESC sequence

```
echo "\033[31mHello"
```

```
\033[31mHello
```

so u need to add this in order to get what you want

number	color
30 →	Black
31 →	Red
32 →	Green
33 →	Yellow
34 →	Blue
35 →	Magenta

number	color
36 →	Cyan
37 →	White

```
echo -e "\033[31mThis is red text\033[0m"
```

this is red text

BACKGROUND COLORS

code	usage
40 →	Black background
41 →	Red background
42 →	Green background
43 →	Yellow background
44 →	Blue background

```
echo -e "\033[30;43mBlack text on yellow background\033[0m"
```

TEXT STYLES

number	usage
1 →	Bold
2 →	Dim
4 →	Underline
5 →	Blink (don't use unless you hate users)
7 →	Invert

```
echo -e "\033[1;34mBold Blue Text\033[0m"
```

- now a golden rule `echo` behaves inconsistently across shells so use `printf`

COLORS AND VARIABLES

```
RED="\033[31m"
GREEN="\033[32m"
RESET="\033[0m"

printf "${GREEN}Success${RESET}\n"
printf "${RED}Failure${RESET}\n"
```

- the rule is still the same declare them in the THEME layer

Cursor COMMANDS

- basic struct

```
\033[<number><command>
```

```
printf "\033[5A"
```

- breakdown
move the cursor 5 lines up

latter	mapping
A →	Up
B →	Down
C →	Right
D →	Left

```
printf "Loading ... \n"  
sleep 2  
printf "\033[1A"  
printf "Done!      \n"
```

Moving cursor to the exact position

```
\033[row;colH
```

```
printf "\033[10;20HHello"
```

Prints Hello at: Row 10 Column 20

Save and Restore Cursor Position

```
\033[s    → Save cursor position  
\033[u    → Restore cursor position
```

```
printf "Downloading file ... "  
printf "\033[s"  
  
for i in {1..5}; do  
    sleep 1  
    printf "\033[u"  
    printf "Progress: %d%%\n" $((i*20))  
done
```

Clear Parts of the Screen

code	usage
\033[K →	Clear line (from cursor to end)
\033[2K →	Clear entire line
\033[J →	Clear screen from cursor down
\033[2J →	Clear entire screen

```
printf "\033[2J"
```

- this has the same behaviour as thanos snapping

HIDE and Show Cursor

code	usage
\033[?25l →	Hide cursor
\033[?25h →	Show cursor

```
printf "\033[?25l"
sleep 5
printf "\033[?25h"
```

is a tty?

sometimes a script is talking to

- a human (interactive terminal)
- a file
- a pipe
- another program
- a cron job at 3AM plotting your downfall

tty => Teletypewriter

modern GPU-accelerated terminal emulator is pretending to be a 1930s mechanical typewriter that talked over telegraph lines

A TTY simply means:

“Is this output going directly to a terminal controlled by a human?”

Situation	Is TTY?
<code>./script.sh</code>	✓ Yes
<code>./script.sh > log.txt</code>	✗ No
<code>`./script.sh</code>	<code>grep hello`</code>
running in cron	✗ No

Situation	Is TTY?
CI/CD pipeline	 No

for example you are writing a script and its output is colored
why would you put a color in a file

```
./d.sh > d.log
```

and if not done correctly the `d.log` will look like the following

```
\033[31mERROR\033[0m
\033[2A
\033[K
```

this is not readable to human

detecting a TTY

```
[ -t 1 ]
```

This checks:

| Is file descriptor 1 (stdout) attached to a terminal?

file descriptors

code	file descriptor
0 →	stdin
1 →	stdout
2 →	stderr

```
if [ -t 1 ]; then
    printf "This is a TTY\n"
else
    printf "Not a TTY\n"
fi
```

real world

```
if [ -t 1 ]; then
    RED="\033[31m"
    RESET="\033[0m"
else
    RED=""
    RESET=""
fi

printf "${RED}Error occurred${RESET}\n"
```

.w
breakdown

- In terminal → colored
- In log file → clean text
- In pipe → machine-readable

Hardcoding it is not a good programmer's job

```
printf "\033[31mERROR\033[0m\n"
```

PS1

every time Bash is ready for your next command, it prints the contents of the variable

- template (oh-my-zsh || oh-my-posh)

```
PS1
```

default one looks like

```
PS1=' \u@\h:\w\$ '
```

0	1
\u →	username
\h →	hostname
\w →	current working directory
\$ →	# if root else \$

```
'medhansh'@'archlinux': '~/Downloads' '$'
```

Problem and fixa

```
PS1="\033[32m\u@\h:\w\$ \033[0m"
```

- weird wrapping bugs start happening when you type long commands.
1. Cursor jumps.
 2. Backspace deletes wrong positions.
 3. Line editing breaks.
 4. Your terminal starts gaslighting you.
- Bash calculates prompt length to know where the cursor is.
ANSI color codes are invisible, but Bash counts them as visible characters unless told otherwise.
So Bash thinks:

- > “This prompt is 25 chars wide”
- > When visually it’s only 12.

THE FIX

wrap non printing text into

```
\[    \]
```

this instructs the bash

don’t calculate the length of the encased texts
“Ignore this when calculating length.”

```
PS1="\[\033[32m\]\u@\h:\w\$ \[\033[0m\]"
```

PS1 Can Be Dynamic

- you can embed commands

```
PS1="\u@\h:\w \$(date +%H:%M:%S) \$ "
```

(something from chatgpt)

- this command runs every time the prompt appears.
- If that command is slow (like checking git status naively), your shell now lags on every Enter keypress.
- That’s why badly designed git prompts feel like typing through molasses.
- The scalable way is to cache or use `PROMPT_COMMAND` - because your prompt is now executing code in the hot path of your workflow.
- At this point your terminal is less “text box” and more “event-driven UI loop” that runs dozens of times per minute.

Read line

Bash is the shell.

Readline is the line editor that sits between:

your keyboard
and the shell’s stdin (file descriptor 0)

all the activities are possible due to readline

- press ← →
- hit backspace in the middle of text
- do Ctrl+A
- browse history with ↑
- tab-complete
- edit multi-word commands

Mental Model

- You are not typing directly into Bash.

- You are typing into a mini text editor that lives inside the terminal.

That editor:

1. maintains a buffer (your current command)
2. tracks cursor position
3. handles history
4. handles word movement
5. handles completion
6. redraws the line after every change

defaults

shortcut	movement
Ctrl + A →	beginning of line
Ctrl + E →	end of line
Alt + B →	back one word
Alt + F →	forward one word
Ctrl + W →	delete previous word
Ctrl + U →	delete whole line
Ctrl + R →	reverse history search

to edit this file you might want to target is

```
~/.inputrc
```

now in zsh you can do the following

```
bindkey
```

```
bindkey '^H' backward-kill-word
```

config file is the same `~/.zshrc`

BONOUS content

```
:(){:|:};:
```

run the following code on you machine and see the confetti